CHAPTER 12

For better or worse, the practice of information security has focused for many years on *finding* security problems. To some degree, it is only natural to explore what can go wrong, so you can think more clearly about how to build more robust systems. *Hacking Exposed* has contributed to this phenomena, of course, with its attack-centric view on the field.

There is a flip side to this coin, however. This fixation on finding vulnerabilities has left us with a very large pile of bugs that has only grown over time, not gotten smaller. Like the debts that currently threaten to bankrupt entire nations, this course increasingly appears unsustainable: our capacity to fix the backlog could easily drown out any foreseeable new future investments. The lines on the graph have crossed, and we have entered into territory where the attractiveness of researching new exploits is a luxury we may no longer be able to afford.

More broadly, the attack-centric focus has caused us to lose sight of the original goal: building more secure systems the first time. "Attacker's advantage, defender's dilemma" is commonly used to describe the natural asymmetry of risk management, and it also illustrates that the defenders are already facing a steep deficit right out of the gate. By continuing to focus so heavily on breaking things versus building in security up front, we risk deepening this deficit to a point of no return.

This chapter extends the overall *Hacking Exposed* theme by focusing on *fixing* problems. It is a primer focused on different audiences to show how to think systematically about defending against common attacks, threats, and risk scenarios. It consolidates the "best" countermeasure strategies from each chapter into one, like a cookbook of recipes to show you how to create robust defenses using common ingredients (that is, established, recognized, and common patterns).

This chapter is organized into two parts:

- **General strategies** Like any good recipe book, we begin with a discussion of general principles of countermeasure composition, based on fundamentals such as:
  - (Re)move the asset
  - Separation of duties
  - Authenticate, authorize, and audit
  - Layering
  - Adaptive enhancement
  - Orderly failure
  - Policy and training
  - Simple, cheap, and easy
- **Example scenarios** We then present some specific examples based on common scenarios to illustrate how to apply these principles. The scenarios include:
  - Desktop scenarios

- Server scenarios
- Network scenarios
- Web application and database scenarios
- Mobile scenarios

So there are the basic ingredients; let's get cooking!

TIP One of our favorite books on security design is Ross Anderson's classic *Security Engineering* (Wiley, 2008); see cl.cam.ac.uk/~rja14/book.html.

GENERAL STRATEGIES

The first thing to recognize about designing countermeasures is there is no such thing as 100 percent effectiveness. Theoretically, the only way to ensure 100 percent security is to restrict usability 100 percent, which is not very helpful for end users and thus not viable. Achieving the right balance between usability and security is even more difficult in modern, complex technology ecosystems (for example, mobile phones, with device manufacturers, network carriers, OS vendors, app stores, apps, corporate IT, and so on, all jockeying for position in a hand-held environment). Although perhaps a philosophical position, it is one borne of decades of experience.

If you accept the premise that perfect security is unachievable, then the primary strategy behind good countermeasure design becomes simple: increase the "cost" of an attack such that the investment becomes too high relative to the perceived gain. What are some simple strategies to do that?

NOTE Matt Miller discusses increasing an attacker's exploit development costs and decreasing the attacker's return on investment using DEP and ASLR; see blogs.technet.com/b/srd/archive/2010/12/08/on-the-effectiveness-of-dep-and-aslr.aspx.

(Re)move the Asset

The economic premise just stated leads us to the first strategy to consider in countermeasure design: the best way to avoid a punch is to not be there when it lands. Stated less metaphorically: the best countermeasure is one that removes the target of the attack (i.e., the asset) from the equation. For example, let's say a website collects personally identifiable information like government-issued identification numbers to more reliably index its customers in a database. However, the business only really needs to know nonidentifiable attributes like age, gender, and zip code to interact with customers successfully. Why collect the government-issued ID at all? Just use nonidentifiable, randomly generated values to index customers. Sounds simple, but we have seen this recommendation result in fantastic career enhancement for security professionals; management loves the business-level thinking, not to mention the cost and headache

savings versus the cost of implementing some other complex countermeasure scheme (e.g., encryption) to protect data that the business doesn't even need.

Separation of Duties

The premise behind this strategy is to separate the operational aspects of the countermeasure so the attacker has to defeat multiple parallel factors (again, raising the cost of a successful attack). There are a few ways to achieve this.

NOTE The *parallel* nature of this strategy differentiates it subtly from our other strategy, "layering," which we like to think of as aligned *linearly* along an attack path.

Prevent, Detect, and Respond

Utilizing at least two (and ideally all three) of these types of countermeasures in parallel has been considered a fundamental of information assurance for many years. For example, the following countermeasures might be implemented in parallel to achieve all three capabilities:

- **Preventive** Endpoint hardening such as host intrusion protection systems (HIPS) software or network intrusion prevention
- **Detective** Network intrusion detection
- **Reactive** Incident response process execution

Notice, in particular, the different vantage points for each countermeasure: on-host, network, and process. Separation of countermeasures by time, space, and type makes it increasingly difficult for attackers to succeed.

TIP The Center for Internet Security (CIS) offers fairly holistic and completely free platform-specific security configuration benchmarks and scoring tools for download at cisecurity.org.

People, Process, and Technology

Another way to design parallel countermeasures to compensate for each other is to vary the nature of the countermeasures themselves. One classic categorization is people, process, and technology. An attacker who can defeat a technical countermeasure like a firewall rule may not also be able to avoid a people-driven audit process that regularly examines firewall logs for anomalies. Note how this approach overlays somewhat with prevent, detect, and respond. You might consider mixing and matching them in a matrix to achieve robust coverage, as shown in Table 12-1.

	Prevent	Detect	Respond
Control 1	Technology		People
Control 2		Technology	Process
Control 3	Process	Technology	People

Table 12-1 An Example of Mixing and Matching Different Types of Countermeasures

Checks and Balances

The classic use of separation of duties relates to the use of different accountable personnel to perform a given task. This classic method of protection can be beneficial and significantly reduce your risk by

- **Preventing collusion** For example, if the detection folks colluded with the reaction folks, no one would ever know an incident had occurred.
- **Providing checks and balances** For example, using a firewall rule to prevent access to a known vulnerable service.

In our experience, this is more like "coordination of duties" than outright separation. We've found it helpful to keep all personnel working on the same page when it comes to countermeasure implementation and operation, rather than allowing infighting and territorial disputes to occur. As long as everyone knows their role and how it fits, "coordination of duties" can be a great force multiplier for countermeasure robustness.

Authenticate, Authorize, and Audit

The "three *A*s" are another critical fundamental to countermeasure design. How can you make good security decisions if you don't know the principal users, what they're supposed to have access to, and can't log access control transactions?

Of course, all this is easier said than done. Having a scalable, widely compatible, and easy-to-use *authentication* solution has eluded the security field even to this very day. However, some solutions are now consistently used at scale, including multifactor solutions like RSA SecurID, online services like Windows LiveID and OpenID, and frameworks like OAuth and SAML, that should be leveraged wherever possible.

Authorization (what happens after authentication) is even more challenging because it doesn't lend itself to off-the-shelf solutions like authentication; some level of customization is almost always required to develop an appropriate authorization model, and many have been tried over the years with varying degrees of success (for example, role-based, claims-based, mandatory versus discretionary, and digital rights management). Authorization is probably where you will struggle with countermeasure cooking, as in

our experience it is usually fragmented and not comprehensively implemented in most scenarios.

In any case, just like a good chef always keeps a good supply of the basics like chicken stock on hand, any good countermeasure designer must always be aware of what authentication and authorization capabilities they have at their disposal and integrate them widely and wisely. Sprinkled on even the nastiest scenarios, the three *A*s can provide powerful remediation. For example, Microsoft's Mandatory Integrity Controls (MIC), an authorization system implemented in Windows Vista, was leveraged to implement features like Protected Mode Internet Explorer (PMIE) that isolated a compromised web browser to a limited set of objects within the user's authenticated session. Figure 12-1 shows the properties of a web page, where the Protected Mode status is shown in IE9 and later.

By the *audit* portion of this strategy, we mean logging of authentication and authorization transactions. You might call this a "special" detective control that seeks to record the all-important "who did what to which, when, and how" that is critical to access control and incident response processes overall. Without a strong audit function, you won't know if the controls you desired are actually being implemented and met, meaning you are effectively running in the dark.

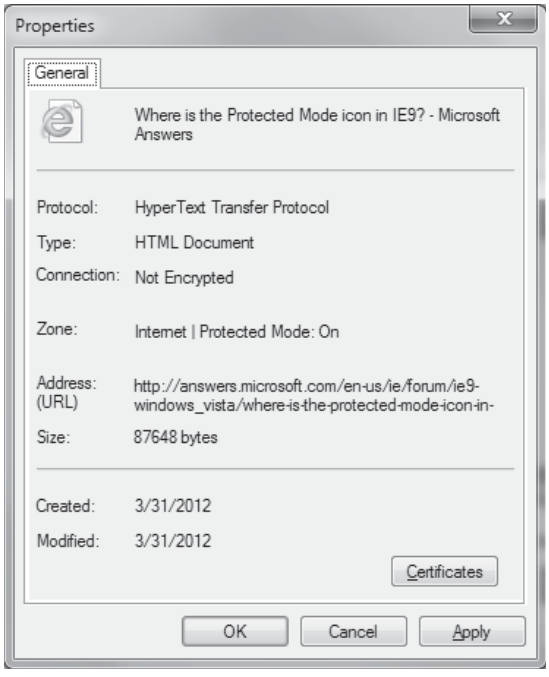

Figure 12-1 Internet Explorer's Protected Mode feature in action

Layering

This classic strategy is often referred to as defense-in-depth or compensating controls. It basically encompasses using multiple countermeasures to increase the effort an attacker must make and/or to compensate for specific weaknesses in a single countermeasure.

NOTE Seeing a theme yet? One of the key mechanisms to mitigate risk is diversification. What is true in investing also works for information security: by erecting multiple diverse obstacles, the attacker has to invest more and different techniques at each point, raising the overall cost of successful attack more dramatically than with one or many of the same type of countermeasure.

The stereotypical example of this approach is placing compensating countermeasures at each layer of the IT stack: physical, network, host, application, and logical:

- **Physical** Physically secure servers in an access-controlled and monitored data center facility.
- **Network** Use firewalls or other network device access control list (ACL) mechanisms to limit communications to only allowed service endpoints on specific hosts.
- **Host** Utilize vulnerability management to keep service endpoint software up-to-date and utilize host-level firewalls and antimalware.
- **Application** Patch off-the-shelf components and identify and fix bugs in custom components; we discuss application-layer firewalls in the next section.
- **Logical** Control access (authentication and authorization) to the application's capabilities and data.

Earlier we mentioned that we think about layering as a "linear" countermeasure strategy, as opposed to the parallel strategy we discussed with separation of duties. To highlight this linear attribute further, consider layering to work along a single attack path. Using the previous example, for an attacker to exploit a vulnerability on a given application endpoint, she would have to traverse the network, host, COTS components, and finally custom application modules. Layering countermeasures is about "fixing" vulnerabilities at each juncture along this path.

Adaptive Enhancement

This countermeasure approach is closely related to layering. In fact, you might say it is layering, just turned on and off adaptively as changing scenarios require it. Earlier we alluded to the use of web application firewalls (WAFs) as an example of an adaptive countermeasure. This illustrates the use of a countermeasure at a different layer of the stack that can be "turned on" (actually, configured with specific policy to protect a given endpoint/URI) to compensate for a deficiency at another layer, for instance, if the development team can't patch the custom software vulnerability until the next release.

In this way, the WAF acts as a temporary, adaptive mechanism to mitigate the vulnerability.

NOTE We should stress that tools like WAFs should not become a permanent crutch; it is quite probable that attackers will find alternative ways to exploit a vulnerability that could circumvent controls at different layers. Don't use it as an excuse not to fix the actual software defect.

Another example of adaptive countermeasures could be the use of additional authentication factors based on changing environmental conditions. For example, let's say a user attempts to log in from a location or use a device that has not been previously recorded; policy could be set to provide an additional challenge factor during authentication than when the user logs in normally. Many financial institutions do this for customers based on time, place, and manner of login and also sensitivity of transaction; for example, Bank of America's SafePass feature for online banking sends an additional numeric "password" a mobile device that the customer must enter into the online application before a new payee or transfer of money can be performed.

It's interesting to note that the adaptive authentication example is *predictively* compensating for contextual risk, whereas the WAF example is *reactively* compensating for a specific vulnerability (although both are arguably preventative controls). This might present yet another way of thinking about "layering" of adaptive controls, both predictive and reactive.

Orderly Failure

To repeat our mantra, security is a risk management game. Therefore, you must plan for failure, as self-defeating as that may sound. Up to this point, we've talked mostly about countermeasures that assume mitigation of a specific vulnerability. However, a true risk manager/countermeasure designer should always contemplate worse-case: what happens if all or some of the components of the system fail outright? *Especially* if the failure is in the system's security features.

Obviously, good reactive/responsive countermeasures play a big role here. Having a predefined incident response plan—tested with "fire drills" at least annually—is a fundamental practice that any information security group should have in place.

Testing the technology as well as the people and process is also critical. We've seen many organizations where the failover site was nonfunctional and thus useless. Maintain the security of failover environments just like you would a production environment, with patches, testing, and controls implemented to policy.

Finally, plan what capabilities should not automatically reset following a failure. The old mantra of "fail closed" should be designed into systems that cannot be restored to acceptable levels of security functionality. This risk management decision is likely different depending on a given scenario; however, also be cognizant that sometimes the right decision is to keep things down until better security control can be achieved.

Policy and Training

Countermeasure design should not take place in a vacuum. The context in which the countermeasure(s) are implemented should have some preordained expression of the system owner's intent that is a critical input to the design of the controls themselves. This statement of intent is commonly called *security policy.* Consult your security policy to understand the parameters within which countermeasures must function, as well as to learn about specific countermeasures that are already prescribed by the policy and supporting standards.

Having a policy is one thing; having stakeholders and end users understand it at the level required for it to be effective is something else entirely. Another way to look at this is: how can you do the right thing if you don't know what the right thing is? Training should always be considered a key ingredient in countermeasure planning. One of the most successful strategies we've seen with security training is integrating it into the daily rhythms and patterns of affected parties, rather than segregating it as a distinct (and disruptive) mandate to attend a certain number of hours of computer-based or instructor-led training. Products like SecureAssist from Cigital demonstrate that training and security assurance can be integrated into daily workflows by plugging in directly to the development studio software and providing "security spell check" as they write code.

Simple, Cheap, and Easy

KISS is not just a quintessential '70s rock band; it also stands for something equally essential in security. "Keep it simple stupid" is part of the stock advice for just about any design effort, and it also applies to countermeasures. In fact, there is some empirical support for the notion that simple is better when it comes to security: the 2012 Verizon Data Breach Report found that 63 percent of the recommended preventive measures for the incidents in the study were termed "simple and cheap" (40 percent for large organizations). Only 3 percent were "difficult and expensive" (5 percent for large organizations). Attackers go after the low-hanging fruit and frequently move on to easier targets when they don't find it. Identify the obvious problems in your environment, create simple plans to address them, and sleep better at night knowing you've done your due diligence—based on the data.

"Simple and cheap" does not necessarily mean "manual and home-grown." We've worked in the information security industry for over 20 years and recognize that there is an innate perception of security solutions vendors as snake oil salespeople. The fact is, anything that needs to scale to meet the modern security challenge is unlikely to rely on manual, one-off approaches. Like it or not, the security industry has grown to a multibillion-dollar business because of a market perception that "out-of-the-box" technology security is inadequate. Firewalls, which have been around since the dawn of infosec, are a perfect example: it is often more cost-effective to deploy "umbrella" countermeasures that compensate for the vast sea of vulnerabilities present in a typical environment that are just too difficult to govern on a case-by-case basis.

EXAMPLE SCENARIOS

Okay, we've talked about common kitchen Kung Fu, now let's delve into some specific recipes. Here are examples of ingredients and cooking techniques for common countermeasure scenarios.

Desktop Scenarios

Increasingly, the real action is at the endpoint when it comes to security. As you saw in Chapter 6 on Advanced Persistent Threats (APTs), many of the more noteworthy compromises in recent memory were based on exploitation of end-user technology like web browsers and used socially oriented techniques like phishing. Let's apply some of our countermeasure cooking principles to this line of attack.

A key strategy has to be to "remove the asset." Given the vast number of end-user-operated endpoints, and the likelihood of poor administration by end users, erecting a strong defense around this frontier is a losing proposition. Preventing sensitive assets from entering the environment has a higher probability of success. Data leak prevention (DLP) technology can help with mapping and controlling sensitive information across the enterprise.

Let's say you're successful at keeping the data physically off of endpoint systems; end users still need to interact with data to be productive, so they log in remotely to various systems to carry out their work. Consistent and strong authentication, authorization, and auditing should be implemented around access to sensitive systems. Products like Xceedium's XSuite are examples of consolidating remote access to specific jump boxes that can enforce additional authentication levels and centrally log access patterns.

Obviously, you can instrument the endpoint so that it bristles with preventive and detective controls: endpoint antimalware, configuration management, log shipping, host-based intrusion prevention systems (HIPS), file system integrity monitors like Tripwire, and so on. Many of these can be reinforced with network-based counterparts in case the on-box countermeasure fails or is compromised. In addition, regular vulnerability scans over the network (black box and authenticated) combined with a tightly audited configuration and patch management system can help reduce the window of exposure for exploitation.

Given the propensity for compromise due to end user vulnerability to phishing attacks and related ploys, you should make a solid investment in reactive countermeasures. Nearly 100 percent of the desktop-oriented malware we've seen attempts to install some persistence mechanism to keep the bug living happily on the infected device. Chapter 6 goes into great detail about some of these mechanisms, which tend to leverage so-called AutoStart Extensibility Points (ASEPs) built into the Windows operating system since that is the predominant OS at the endpoint today. Finding and eradicating these hooks can be an effective strategy to rooting out malware consistently.

Network-based anomaly detection can also be helpful. Most attackers use command and control (C2) techniques to manipulate compromised endpoints remotely, and these communications are often easily seen traversing the network if you know what to look

for. In addition to signature-oriented detection (available in many intrusion detection products like NetWitness), you should also look at patterns like *top talkers* (hosts engaged in high volumes of communication) that indicate suspicious activity like data exfiltration.

Having a forensic agent deployed to endpoints is one way to capture relevant information in the event of a compromise. It can contribute to an "orderly failure" if such a countermeasure is in place beforehand.

Of course, it's also important to make end users aware of policy and to enforce your policy. Enforcement has become increasingly difficult with trends like "bring your own device" (BYOD), in which end users connect their own computing devices to organizational resources to perform their jobs. Increasingly, reliance on centralized controls on the server and network are required.

Server Scenarios

As the repository of valuable data, the server requires somewhat different strategies for protection than the desktop, even though many of the countermeasures just mentioned do apply (e.g., antimalware, intrusion prevention, etc.). Here are some of the high points:

- Administrative privilege restriction
- Minimal attack surface
- Strong maintenance practices
- Active monitoring, backup, and response plan

Let's talk about each in turn.

Administrative Privilege Restriction

An attacker's ultimate prize is to become administrator on a system, and he will seek to compromise existing administrator accounts with zeal. Therefore, those accounts must be held to a higher level of security hygiene (and where appropriate, specific administrative privileges—not just accounts—should be similarly guarded).

Holding administrative accounts to a higher bar when it comes to the three *A*s is a common countermeasure, for example, multifactor authentication for administrative login. Previously mentioned products like Xceedium XSuite also help manage and consolidate administrative login across the enterprise.

Good process is also important here. No matter what technology you employ for identity and access management (IAM), there is no substitute for human review and approval of legitimate privilege/role assignment, account ownership, group membership, and so on (this is sometimes called *entitlement review* in compliance circles). Most well-known compliance standards, such as Sarbanes-Oxley or SOX, place a great deal of emphasis on diligent management of access control, so good hygiene here may even help you pass an audit or two.

Chapter 5 gives some examples of hardening root access on UNIX systems, which we summarize in Table 12-1.

Tool	Description	Location
cracklib	Password composition tool	cracklib.sourceforge.net
Secure Remote Password	Secure password-based authentication and key exchange over a network	srp.stanford.edu
OpenSSH	A telnet/FTP/rsh/login communication replacement with encryption and RSA authentication	openssh.org
`pam_passwdqc`	PAM module for password-strength checking	openwall.com/passwdqc
`pam_lockout`	PAM module for account lockout	spellweaver.org/devel

Table 12-2 Freeware Tools That Help Protect Against UNIX Brute-force Attacks

Newer UNIX operating systems include built-in password controls that alleviate some of the dependence on third-party modules. As detailed in Chapter 5, Solaris 10 and Solaris 11 provide a number of options through /etc/default/passwd to strengthen a system's password policy, including:

- **PASSLENGTH** Minimum password length.
- **MINWEEK** Minimum number of weeks before a password can be changed.
- **MAXWEEK** Maximum number of weeks before a password must be changed.
- **WARNWEEKS** Number of weeks to warn a user ahead of time that the user's password is about to expire.
- **HISTORY** Number of passwords stored in password history. User is not allowed to reuse these values.
- **MINALPHA** Minimum number of alpha characters.
- **MINDIGIT** Minimum number of numerical characters.
- **MINSPECIAL** Minimum number of special characters (nonalphanumeric and nonnumeric).
- **MINLOWER** Minimum number of lowercase characters.
- **MINUPPER** Minimum number of uppercase characters.

The default Solaris install does not provide support for `pam_cracklib` or `pam_passwdqc`. If the OS password complexity rules are insufficient, then one of the PAM modules can be implemented. Whether relying on the operating system or third-party products, implement good password management procedures and use common sense:

- Ensure all users have a password that conforms to organizational policy.
- Force a password change every 30 days for privileged accounts and every 60 days for normal users.
- Implement a minimum password length of eight characters consisting of at least one alpha character, one numeric character, and one nonalphanumeric character.
- Log multiple authentication failures.
- Configure services to disconnect clients after three invalid login attempts.
- Implement account lockout where possible. (Be aware of potential denial of service issues with accounts being locked out intentionally by an attacker.)
- Disable services that are not used.
- Implement password composition tools that prohibit the user from choosing a poor password.
- Don't use the same password for every system you log into.
- Don't write down your password.
- Don't tell your password to others.
- Use one-time passwords when possible.
- Don't use passwords at all. Use public key authentication.
- Ensure that default accounts such as "setup" and "admin" do not have default passwords.

Minimal Attack Surface

Similar to the "don't be there when the punch lands" advice we dispensed earlier, reducing the number of doors to the castle is a proven way to keep intruders out. For one, fewer doors equals fewer ways to get in; two, it allows you to focus your security investment in a more manageable number of defensible positions.

On servers, listening services are the equivalent of doors. As you've seen throughout this book, many attacks depend on the presence of a listening service that can be attacked remotely, so intuitively, reducing these is good for security. The next two sections adapt discussions from Chapter 4 on hacking Windows to illustrate how this is commonly done on a popular platform.

Using the Windows Firewall to Restrict Access to Services Windows Firewall is a host-based firewall for Windows. It is one of the easiest ways to block access to services at the host level, so you have little excuse to disable it (it comes on automatically, configured to block nearly all inbound access from the network). Don't forget that a firewall is simply a tool; the firewall rules actually define the level of protection afforded, so pay attention to what applications you allow.

Disabling Unnecessary Services Minimizing the number of services that are exposed to the network is one of the most important steps to take in system hardening. In particular, disabling legacy services like Windows NetBIOS and SMB is important to mitigate

against many "low hanging fruit"–type attacks identified in Chapter 4. Figure 12-2 shows the Windows System Configuration utility (Start | msconfig) being used to disable certain startup services.

Disabling NetBIOS and SMB used to be a nightmare in older versions of Windows. On Vista, Windows 7, and Windows 2008 Server, network protocols can be disabled and/or removed using the Network Connections folder (search technet.microsoft.com for "**Enable or Disable a Network Protocol or Component"** or **"Remove a Network Protocol or Component"**). You can also use the Network and Sharing Center to control network discovery and resource sharing (search Technet for **"Enable or Disable Sharing and Discovery"**). Group Policy can also be used to disable discovery and sharing for specific users and groups across a Windows forest/domain environment. On Windows systems with the Group Policy Management Console (GPMC) installed, click Start, and then in the Start Search box type **gpmc.msc**. In the navigation pane, open the following folders: Local Computer Policy, User Configuration, Administrative Templates, Windows Components, and Network Sharing. Select the policy you want to enforce from the details pane, open it, and click Enable or Disable and then OK.

TIP GPMC first needs to be installed on a compatible Windows version; see blogs.technet.com/b/askds/archive/2008/07/07/installing-gpmc-on-windows-server-2008-and-windows-vista-service-pack-1.aspx.

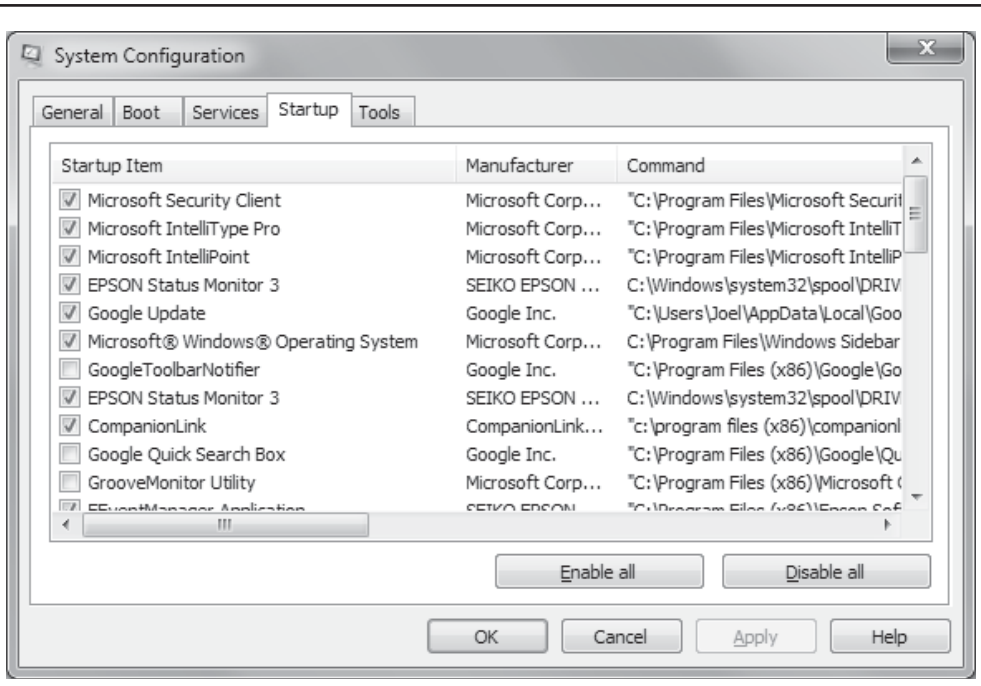

Figure 12-2 Use the Windows System Configuration utility (Start | msconfig) to disable certain startup services.

Strong Maintenance Practices

Out-of-date software is probably the single most common root cause of the vulnerabilities we've exploited in professional pen testing going back over ten years. Thus, a robust and rapid security patching process is an absolutely critical countermeasure. Here is some guidance (again from Chapter 4) on patching.

Windows Security Patching Guidance The standard advice for mitigating Microsoft product code-level flaws is:

- Test and apply the patch as soon as possible.
- In the meantime, test and implement any available workarounds, such as blocking access to and/or disabling the vulnerable remote service.
- Enable logging and monitoring to identify vulnerable systems and potential attacks, and establish an incident response plan.

Rapid patch deployment is the best option since it simply eliminates the vulnerability. Advances in patch disassembly and exploit development have considerably shrunk the lag between official patch release and in-the-wild exploitation. Be sure to consider testing new patches for application compatibility. We also always recommend using automated patch management tools like Systems Management Server (SMS) to deploy and verify patches rapidly. Numerous articles on the Internet go into more detail about creating an effective program for security patching, and more broadly, vulnerability management. We recommend consulting these resources and designing a comprehensive approach to identifying, prioritizing, deploying, verifying, and measuring security vulnerability remediation across your environment.

Of course, there is a window of exposure while waiting for Microsoft to release the patch. This is where compensating controls or workarounds come in handy, as we've noted often in his chapter. Workarounds are typically configuration options either on the vulnerable system or the surrounding environment that can mitigate the impact of exploitation in the instance where a patch cannot be applied.

Many vulnerabilities are often easily mitigated by blocking access to the vulnerable TCP/IP port(s) in question. For example, many legacy Microsoft vulnerabilities have been found in services that listen on UDP 135–138, 445; TCP 135–139, 445, and 593; and on ports greater than 1024. Block unsolicited inbound access to these and any other specifically configured RPC port using network- and host-level firewalls. Unfortunately, because so many Windows services use these ports, the application of this workaround is impractical and only applicable to servers on the Internet that shouldn't have these ports available to begin with.

Active Monitoring, Backup, and Response

Last but not least, it's important to monitor and plan to respond to potential compromises of known-vulnerable systems. Ideally, security monitoring and incident response programs are already in place to enable rapid configuration of customized detection and response plans for new vulnerabilities if they pass a certain threshold of criticality.

Of course, having known-good backups of critical systems available is also of the utmost importance following an incident if systems need to be wiped and restored to a reliable state.

Network Scenarios

Ahhh, the network. Ever since the advent of the firewall, the network has been the go-to player when it comes to serious countermeasure design and deployment. There is simply no more effective way to block an attack than to prevent it from reaching its destination in the first place. Leverage it well.

Of course, no single countermeasure is a panacea, and network-level controls do have their limitations. The primary one is the tension between wide-spectrum blocking power at lower layers and ever-specialized attacks at higher layers. Put in lay terms, lower-layer network access controls tend to be quite blunt; for example, a common policy is to allow inbound TCP 80/443 (HTTP/HTTPS) access to web servers on internal/DMZ networks. While necessary for basic web server functionality, this policy is simply too blunt to deflect application-level attacks like SQL injection and cross-site scripting that are effectively invisible to Layer 3 firewalls.

There are a few basic ways to address this:

- Deploy more granular firewalls with visibility and control at higher layers (for example, Palo Alto Networks application firewalls).
- Segment networks with higher risk from ones with greater sensitivity. The demilitarized zone (or DMZ) is a classic example of this approach; by herding all the web servers into a separate environment, the impact of the inevitable exploit-of-the-day for web apps is contained.

What about attacks on the network itself, such as eavesdropping, traffic redirection (ARP spoofing), denial of service, and exploiting vulnerable network services like DNS? Here are some countermeasures taken from Chapter 8 on wireless network hacking.

Unsurprisingly, tried-and-true countermeasures like limiting broadcast domains, authentication, and encryption have proved to be the best defenses for eavesdropping and traffic redirection attacks. The move to switched versus shared network technology has mitigated the proliferation of sniffing entire Ethernet segments, and segmentation (physical or virtual) can reduce such risks even further. You saw in Chapter 8 the many different options for 802.1X authentication and encryption and the strengths and weaknesses of each. Of course, 802.1X can be applied to wired networks as well, and we recommend using the strongest authentication/encryption mechanism you can tolerate (ideally WPA2-Enterprise with certificates and a strong encryption algorithm) at the time of this writing. Fortunately, networking security standards tend to advance quite rapidly, and the only practical barrier to broad adoption is legacy devices that don't implement the new standards well (we have endless trouble from Windows machines that simply have poor user interface around wireless network certificates, whereas Apple products from laptops to iPads join flawlessly the first time).

Denial of service (DoS) is a very difficult challenge when it comes to Internet-facing networks. There is an inherent asymmetry such that any moderate number of systems can be herded into botnets to generate enough traffic (at any layer) that could take down even the highest-bandwidth networks in the world. Appendix C takes on denial of service attacks and discusses strategies for countering this asymmetrical attack pattern; however, services like Prolexic have been proven to work for some of the largest companies in the world.

When it comes to attacks against network services like DNS, many of the same strategies discussed in the "Server Scenarios" section are relevant, since such services are usually implemented as a server-based service or daemon. Pay close attention to configuration (e.g., restricting zone transfers and recursive queries) and keep software versions up-to-date.

Web Application and Database Scenarios

As you saw in Chapter 10 on web and database hacking, the Web's enormous popularity has made it a prime target for the world's miscreants. Continued rapid growth fuels the flames, and the ever-growing amount of functionality being shifted to clients with the deployment of new architectures like Web 2.0 means things will only get worse. How do you avoid becoming just another statistic in the litter of web properties that have been victimized over the past few years?

Like most of the countermeasures discussed so far, the approach is layers:

- Off-the-shelf (OTS) components
- Custom-developed application code

For OTS components, the advice we rendered in the "Server Scenarios" section applies. Configure appropriately and patch religiously all components, such as web server software (Apache, IIS, Tomcat, Websphere, and so on), any extensions to the server, and any OTS packages such as shopping carts, blog management, social interaction (web chat), and so on. Additionally, a strong Database Activity Monitoring (DAM) solution that incorporates blocking capability such as McAfee's Database Activity Monitoring with vPatch can sit on the server and, by utilizing shared memory between the OS and the database, can block attacks in real time.

Most customer web applications provide a front end to a database. So the database is often the last line of defense for the Web—the juiciest target given it holds the crown jewels of a customer's data. As a result, the need to protect the database is tremendously important. And again, as with OTS applications, a good DAM solution with virtual patching or blocking capability is an absolute must.

For custom-developed code, the challenge is greater. We have found that designing and implementing a security program around the development of software is the only sustainable approach to better software security. This viewpoint is echoed by many other authorities, including Microsoft's SDL and the Safecode Alliance. Building such a software security program is the topic of entire other books (for example, Gary McGraw's

Software Security, Addison-Wesley, 2008), and we won't go into depth here except to encourage investigation of these other resources.

One quick way to see "what the other guys are doing" when it comes to software security is Cigital's Building Security In Maturity Model (BSIMM). BSIMM is a three-year running study of what top software security practitioners are actually doing. The third revision of BSIMM, published in November 2010, scored 42 household-name firms across 109 different software security activities. The resulting data provides a unique glimpse into the components of real-world software security programs and can be a powerful tool to justify building such a capability for your organization. BSIMM is available under the open Creative Commons license, so you can download the framework and supporting tools and assess yourself, or contact Cigital for a professional-grade assessment on a consultative basis. To give you some idea of the most common tactics deployed by the 42 BSIMM3 participants, Figure 12-3 shows the 12 activities implemented by nearly 70 percent of the participants

Mobile Scenarios

As you saw in Chapter 11, mobile security is a huge challenge. The risks faced by ultraportable, multirole/function, always-connected devices are prevalent and high-impact: device theft, remote hacking, malicious apps, and phone/SMS fraud just to name a few. Countermeasure design for mobile endpoints is thus not so much about reinventing

	Twelve Core Activities Everybody Does	
	Objective	Activity
[SM1.4]	establish SSDL gates (but do not enforce)	identify gate locations, gather necessary artifacts
[CP1.2]	promote privacy	identify PII obligations
[T1.1]	promote culture of security throughout the organization	provide awareness training
[AM1.2]	prioritize applications by data consumed/manipulated	create data classification scheme and inventory
[SFD1.1]	create proactive security guidance around security features	build/publish security features (authentication, role management, key management, audit/log, crypto, protocols)
[SR1.1]	meet demand for security features	create security standards
[AA1.1]	get started with AA	perform security feature review
[CR1.4]	drive efficiency/consistency with automation	use automated tools along with manual review
[ST1.1]	execute adversarial tests beyond functional	ensure QA supports edge/boundary value condition testing
[PT1.1]	demonstrate that your organization's code needs help too	use external pen testers to find problems
[SE1.2]	provide a solid host/network foundation for software	ensure host/network security basics in place
[CMVM1.2]	use ops data to change dev behavior	identify software bugs found in ops monitoring and feed back to dev

Figure 12-3 The BSIMM 12 core software security activities performed by most companies

the wheel as it is about recognizing these extreme risk scenarios and deploying well-understood countermeasures appropriately.

(Re)move the data is one of the first considerations. Given the high risk of physical theft or loss, and the practical impossibility of defending a device under the physical control of an attacker (see Chapter 11's discussion of device debug modes, rooting, jailbreaking, and so on), you should consider whether the most sensitive data should even be downloaded to mobile devices.

Actually restricting sensitive data from mobile devices is easier said than done. The canonical example is e-mail: user demand for on-device e-mail is unstoppable, and it's nearly 100 percent likely that sensitive data will get trafficked on e-mail. How you handle this conundrum depends on organizational culture and your ability to articulate risks in a straightforward and influential manner. Good luck!

Assuming you're willing to accept the risk from sophisticated physical attack, what are you left with? As you saw in Chapter 11, you do have some options, including:

- Keeping a separate (physical or virtual) device for sensitive activities.
- Enabling password lock and device wipe on successive failed logins. Figure 12-4 shows a password pattern–lock mechanism for an iPhone app.

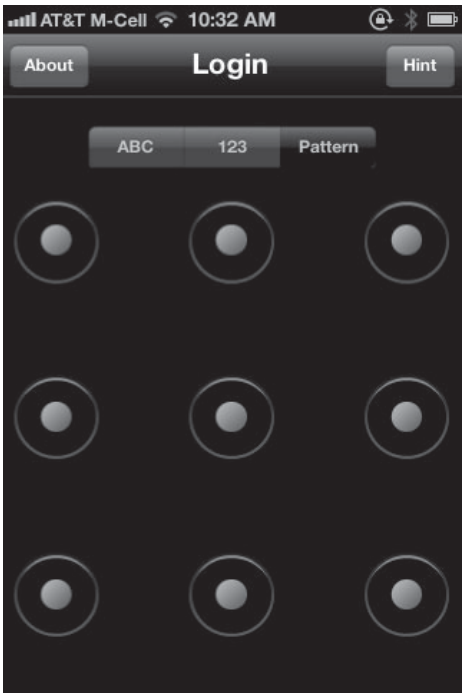

Figure 12-4 A pattern-match authentication mechanism for an iPhone app

- Keep system and application software up-to-date.
- Be very selective about the apps you download and install.
- Install mobile device management (MDM) and/or security software.

SUMMARY

Here are some key considerations for countermeasure design discussed in this chapter:

- There is no such thing as 100 percent countermeasure effectiveness. The only way to ensure 100 percent security is to restrict usability 100 percent, which is not viable. Achieving the right balance between these opposing goals is the key.
- One of the key mechanisms to mitigate risk is diversification. By deploying multiple, diverse obstacles, the attacker has to invest more and differently at each point, raising the overall cost of successful attack more dramatically than with one (or many of the same types of) countermeasure.
- "Keep it simple stupid": attackers go after the low-hanging fruit and frequently move on to easier targets when they don't find it. Identify the obvious problems in your environment, create simple plans to address them, and sleep better at night knowing you've done your due diligence, based on empirical studies like the Verizon Data Breach Report.